CIS 1921 Final Project Description

Project Overview

The goal of the final project is to deepen your understanding of concepts we have covered in the class by exploring a problem of your choosing. **Problems must be completed in pairs of students.**

Please see the **Project Ideas** section at the bottom for some inspiration.

IMPORTANT: Late days CAN NOT be used on any of the deadlines of the final project.

Project Timeline and Grading Breakdown

You cannot use late submissions for any of the final project deadlines.

- **March 3** — Project partners due
  - 3% of grade
- **March 31** — Project proposal due
  - 10% of grade
- **By April 14** — Project check-in due (schedule meeting with TAs before April 14 or written check-in due April 14)
  - 12% of grade
- **April 28** — Project presentations (in class)
  - 15% of grade
- **April 28** — Final project submission due
  - 60% of grade

Choosing a Project

While you and your partner have near-total freedom in selecting a topic for your project, it must be clearly related to the overarching theme and material of the course. Although it is encouraged to use Python and PicoSAT/OR-Tools, you are welcome to use other tools if there is a good reason.

See the Project Ideas section at the end of this document if you need some inspiration.
Your project must be one of the following categories of projects:

- **Application Projects –** A more applied project seeks to utilize the tools we have learned (PicoSAT, LP, MIP, CP, etc.) in order to effectively model and solve a challenging

real-world problem. You may choose to build an application with user-friendly features (GUI, web app, etc.) however, <u>it is your responsibility to ensure that the majority of your effort is put on the course-related subject programming</u>. That is, don't spend the majority of your time building a fancy frontend/backend – it is okay for this to be scrappy. You should experiment with multiple varieties of the problem or allow customization of constraints, etc. Refer to the section below titled "Projects from Past Semesters" to get a better understanding of what an application project would look like.

- Past application projects have seen students write over 1000 lines of code, with over 70% of it being SAT/LP/CP-related. You do not *need* to hit 1000 lines; this is purely just a reference to what past groups have done.
- Some of the most successful application projects were those that experimented with tougher constraints (even if they weren't immediately applicable to the project). We value experimentation with some of the more advanced techniques we have covered in class!

- **Theoretical Projects –** A theoretical project seeks to expand our knowledge about some of the high-level techniques we've learned, or even develop new techniques. Read a recent research paper in the fields of the course and write some questions you had. Come up with initial predictions to what the answers to the questions would be, then code up a Jupyter notebook that tries to answer these questions with clear annotations describing what you are trying to accomplish with the code. Lastly, compare the results of the implementation with your hypotheses.
    - While we will not set a hard limit on the lower bound or upper bound of the number of questions you need to develop, it is your responsibility to ensure that the questions you come up with are not too trivial and not too difficult. You don't want to have questions that can be answered with 2 lines of code, and you don't want to have questions that can be answered with hundreds of lines of code.
    - We expect answering all of the questions you pose to take around 200-400 lines of code, but this is not a required range.
    - You will get guidance on the questions/hypotheses you pose during the project check-in.
    - Remember, it's OK if your project does not end up giving an improvement over the baseline, as long as it was a reasonable idea. We would much rather see a project that discredits an interesting proposal, ideally with some analysis of why it may not have worked, rather than a project whose results cannot be reproduced or which is successful only on a couple cherry-picked examples. In essence, **explain your thought process.**

IMPORTANT: Do note that a sizable portion of your grade is allocated to the novelty of your project idea, as we want to encourage a diverse and creative range of projects across the class. There are several ways you can satisfy this novelty requirement:

Option 1: You may choose to explore a more theoretical project or solve a problem related to graph optimization, network optimization, or classic combinatorial problems like the Traveling Salesman Problem (TSP). If you have any other theoretical project ideas, please feel free to ask any of the staff, and we would be happy to discuss their feasibility and help you refine your approach.

Option 2 (choose at least three):

- Real-world dataset integration (e.g., using external APIs or datasets to ground your problem)
- Multiple solver implementations (e.g., comparing SAT vs. CP vs. MIP approaches). We'll expect you to investigate performance and what makes certain solver better than others
- Parameterized instance generator (creating scalable/randomized test cases to stress-test your model)
- User-configurable constraint systems (allowing users to modify constraints dynamically)
- Use multiple techniques e.g TSP + CP, CP + ML, SAT + heuristic search etc
- If this applies to your project, additionally add a generator for you "puzzle" do an analysis or what makes generated puzzles hard

If you have another creative direction in mind, feel free to reach out to us - we are happy to discuss and approve other novel ideas.

In addition to novelty, a large portion of your grade will be based on rigor, which should be demonstrated clearly in your report. This includes thorough experimentation, well-designed tests, performance evaluation, use of multiple solvers where appropriate, and clear visualizations (graphs, charts, etc.). Strong projects not only work but also carefully analyze why they work (or don't).

Partner Selection (due March 3 by 11:59 pm)

It is your responsibility to find a partner for your project. A good time to do this would be before/after class. If you need help finding a partner, feel free to post on Ed so your classmates can see, or send the teaching staff a message. We recommend partnering with someone that you don't already know – meet someone new and make a new friend!

Once you have found a partner, simply submit a blank page (or your favorite meme) on Gradescope as a group submission with your partner and yourself.

Project Proposal (due March 31 by 11:59 pm)

You and your partner must jointly submit one project proposal document containing the following information:

- The names of both group members
- A title for your project
- A 1 to 3 paragraph description of the project
    - What is it?
    - What are the goals of the project?
    - How does it relate to the course material?
    - *If theoretical, give us an overview of the paper you want to focus on, and provide us the link to the paper.*
    - ***How do you plan to achieve novelty points? (if using a dataset, provide a link)***
- A 1 to 3 paragraph description of your tentative plan for implementing the project
    - What tools or techniques will you apply?
    - What resources do you plan to use for this project (i.e. source inspiration, articles you read, etc…)
    - *If theoretical, explain the reasoning for choosing this paper – what questions did you have when trying to search for papers, and why did you settle on this one?*
    - *If possible, list some potential topics you will explore/base your questions off of, and list some of the questions of interest that you came up with when looking over the paper.*
- A 1 to 3 paragraph description of how your project should be evaluated
    - How will you test it?
    - What will be your metrics for success?

The project proposal must be uploaded to Gradescope as a group submission with both partners.

Project Check-in (due April 14 by 11:59 pm)

At this point, you should have implemented a significant amount of the functionality of your project. As an estimate, you should feel that your project is at least roughly 75% complete based on your own understanding of the project's goals and evaluation metrics (from experience, you are usually never as close to "done" as you think you are).

For this checkpoint, you have two options of how you can complete it:

Option 1) Online Demo with Course Staff (Recommended)

Schedule a Zoom meeting with a course staff member to showcase the code and functionality you've developed so far (for application) or discuss the research behind your questions and hypotheses (for theory). No slides are required for this meeting.

- When: The meeting should be scheduled for 15 minutes during office hour slots, before April 12th (inclusive).
- How: Email the **instructors and TA** beforehand to secure an exact time slot and minimize wait time in the queue. If you cannot attend any office hour slot, please proceed with Option 2.

Option 2) Written Submission

If you are unable to attend the office hour slots or prefer a written submission, both partners must submit a single project check-in document. The document should answer the following criteria (1-3 pages is recommended):

1. Names of both group members.
2. Project title.
3. Overview of progress so far – include data, results, or screenshots if possible (code files are not needed, but short snippets are acceptable).
4. Explanation of any changes to the scope or goals of the project, and the reasons for those changes.
5. Overview of remaining tasks and your plan to complete them.

Submission: The project check-in must be uploaded to Gradescope as a group submission with both partners listed.

Project Presentation (due April 28 in class)

You and your partner will jointly give a 5-minute presentation to the class demonstrating the functionality or end-result of your project. Please save a minute to allow for questions from the audience.

Given the limited time, please focus your presentation on three key components:

1. **Project formulation** – the problem you are solving and your approach
2. **(IMPORTANT) Live demo (if applicable)** – a brief demonstration of your system
3. **Key results** – the most important findings

You may briefly mention potential next steps or improvements, but do not spend too much time talking about all the nitty-gritty implementation details (save that for the write-up). You'll be graded on the quality of work accomplished, presentation quality, being respectful/paying attention to other presentations, etc.

Final Project Submission (due April 28, by 11:59 pm)

You and your partner must jointly submit the following **in a zipped folder** named `1921Project_Firstname1_Firstname2.zip`, uploaded to Gradescope as a group submission:

Theory Project Deliverables:

- All code, notebooks, and research papers related to your final project.
    - Make sure to include all data files relevant to your project, if applicable.
    - Please include **all** relevant research papers – this should include the overarching research paper you read, as well as any other papers you reference throughout the project.
- README document explaining:
    - An overview of the file structure and an explanation of what each file contains.
    - If you have just one code file, just summarize what it accomplishes.
    - Provide thorough instructions on how to run your final project.
        - You should also describe how to run any test suites, outline necessary dependencies, etc. Please spend a good amount of time ensuring this information is thorough, as being unable to run your project may lead to a point deduction.
- Video demo (1-2 minutes)
    - Record a short video running your code. Allow time for necessary outputs to be shown on the video.

    - You don't need to talk or explain anything during the video; just show functionality of the code.
- Annotations
    - All code must be thoroughly annotated. You don't need to comment every single line of code.
    - Comments can be for each "section" of code, and provide a short (1-2 sentence) description of the code that follows.
- Report
    - The majority of your time should be spent on writing your project report. As you write this, you should pretend that someone with no industry knowledge regarding your problem/project is reading your report. You can assume the reader is familiar with class-related topics. You don't have to spend time discussing the theory of CP or SAT, for example, but should spend time introducing/defining any new terms related to your project.
    - In this document, you should provide a summary/abstract of the overarching research paper that you read.
    - Walk through the main ideas of the paper, discuss their importance, relevancies, and implications.
    - Include graphs, charts, visuals, etc… These should be supplemented with an explanation of what it is, as well as what conclusions you draw from it.
    - The second part of this document should include the questions you explored in your project.
        - For each question, present the question itself, the hypothesis you had for the question, and finally a short analysis of if your hypothesis was supported by or rejected by your investigations and why.
        - Your paper should read like a white paper and transition naturally between questions/sections.
        - We expect to see a couple of paragraphs for each question.

Application Project Deliverables:

- All code, notebooks, and research papers related to your final project.
    - Make sure to include all data files relevant to your project, if applicable.
- Report:
    - The majority of your time should be spent on writing your project report. As you write this, you should pretend that someone with no industry knowledge regarding your problem/project is reading your report. You can assume the reader is familiar with class-related topics. You don't have to spend time discussing the theory of CP or SAT, for example, but should spend time introducing/defining any new terms related to your project.
    - An overview of what you have completed in the final project. Write this as if the person reading it is completely unfamiliar with your project.

        - Describe what the problem statement is, your initial approach to solve the problem, the issues you ran into while developing the project, how you adapted, etc.
        - Briefly analyze your solver. Does it perform as expected, and why? Is anything missing? Does it take too long to run, and why?
        - Any graphs, charts, visuals you provide should be supplemented with an explanation of what it is, *as well as what conclusions you draw from it.* Feel free to make hypotheses as to why you observe what you do, and supplement them with explanation. *I'd rather you do thorough analysis and be wrong about your attempts at explaining your observations than to not do any analysis at all.*
        - This overview should be roughly 1-2 pages.
    - Explanation of the code
        - Use this as an opportunity to supplement the comments you left in the code. If you think it is necessary to explain something in more detail, do that here. If you need to include diagrams, do that here.
    - An overview of the file structure / explanation of what each file contains.
    - Thorough instructions describing how to run your final project. You should also describe how to run any test suites, outline necessary dependencies, etc. Please spend a good amount of time ensuring this information is thorough, as being unable to run your project may lead to a point deduction.
    - Spend time talking about all the experimentation you did. Make hypotheses about why some things worked better than others. Do your best to try to answer your hypotheses with the results of your project. But it is also fine to attempt to answer your hypotheses with a claim, even if it happens to be incorrect.
    - Discuss any testing you did.
- Video demo (1-2 minutes)
    - Record a short video running your code. Ideally, take us through an example. That is, provide an example instance of the problem you were trying to solve, then run the solver, and finally show the output.
    - You don't have to talk during the video if you do not wish to, but it then must be very clear what we are watching.
- Annotations
    - All code must be well-annotated. The goal of good annotations is for us to understand your thought process without having to ask you to walk us through everything. In general, overannotation is better than underannotation.
- **Your final report should be "professional" in terms of formatting. I'd advise you to not use bullet points and instead use paragraphs of writing.**
    - You will **not** be graded on grammar, English, or length for the project. That is, do not worry about making sure your sentence structure is correct. Focus more on providing information about your project, even if you deem it to be unnecessary. I'd rather read a very long document that details all of the thoughts and decisions you made throughout the project (with respect to implementation, pivoting,

analysis, etc.) than a shorter document that is well formatted but omits the important details.
    - You will be graded on quality of work submitted
        - With respect to code, we won't give much attention to syntax, formatting or organization. We will pay attention to the section/percentage of code dedicated to class-related concepts.
        - With respect to the writeup, you'll be graded on the quality of information you provide. Special attention will be given to your analyses and breakdown of thoughts/decisions made.
        - We expect there to be performance metrics for your project. You'll be graded on the analysis of your performance. What we are looking for is an analysis of *why* your tool/theory performs/behaves the way it does. If your tool is performing below expectations, or the theory you worked on didn't give the results you expected, mention that, and explain why you think it is performing that way.
- Any other files that you feel add necessary context for your project

Project Ideas

Projects from Past Semesters:

- Hospital Scheduler – Hospitals have many different types of shifts, and employees need to make sure they complete certain types of shifts. Mount Sinai currently uses a pencil-and-paper system to build schedules for their employees. The team built a schedule-creation tool, where users can specify the employees, their availability, and other soft constraints to produce the most optimal solver.
- NBA Roster Optimization – Basketball rosters can be built with an underlying motif, such as prioritizing perimeter shooting, defense, small-ball, etc. NBA teams also need to abide by the salary cap. Teams also want to make sure their lineup isn't too old or too young. This project build a roster optimizer, which addressed the many constraints involved with building a team.
- Cookin' Not Cooked – This team used Constraint Programming to build a meal planner. The user inputs the ingredients they have, the associated quantities of each ingredient, and auxiliary information such as allergies, and the solver gives a meal suggestion that abides to the constraints of calories, carbs, etc.
- Numberlink Solver/Puzzle Generator – The team built a simple UI where users can input a numberlink game (either through file input or via clicking), and the tool would solve the puzzle. They experimented with different methods of solving the puzzle – using edge-based SAT encoding, path-based SAT encoding, Constraint Programming, etc.
- Optimal Portfolio Investments – The team created a tool where users could specify the amount of money to invest, the different sectors they wanted to invest in (e.g. energy, technology, etc.), percentages of how much of the investment should be allocated to each sector, and more. After this input, the solver would call the Yahoo Finance API to get stock data for about 100 stocks, and use that to create an investment portfolio.

Some more Inspiration

This non-exhaustive list of ideas is meant to act as inspiration. We encourage you to **not** simply copy one of these project ideas, but at the very least, find a way to enhance the idea into something even better!

- **(**) Pick one or more of the problems on CSPLib, a public database of constraint programming problems, and attempt to model and solve it effectively in an applied or experimental project.**
- Pick a different puzzle game than Sudoku and build an application to play it using SAT, MIP, or CP in an applied project.
- Formulate a real-world scenario as packing, planning, or scheduling problem and build an application to solve it using SAT, MIP, or CP in an applied project.
- Pose your own interesting CP or MIP problem, attempt to model and solve it effectively, and investigate experimentally what makes an instance of the problem hard or easy.
- Build a basic package manager using SAT or CP for dependency resolution (you may also want to take a look at the CUDF format for package management).
- Use constraint programming for procedural content generation of 2D maps by solving constraints about which tiles can appear next to each other (here's a cool blog post).
- Invent or modify, implement, play around with, and rigorously test a variety of decision heuristics to see how they affect the performance of PennSAT in an experimental project.
- Expand PennSAT to use some of the modern SAT solving techniques we learned, in particular 2-Watched Literals and Conflict Driven Clause Learning (CDCL). Just 2WL + CDCL would definitely be large enough in scope — this might be a tricky project, so please reach out to me for advice and resources if you're considering it.
- Build your own CP solver inspired with lazy clause generation using PicoSAT as a backend.
- Use the CryptoMiniSAT solver to implement an attack on a cryptographic cypher (this may be difficult if you have no background in cryptography).
- Develop your own scheduling tool for an organization of your choosing.
- Knowledge Graph Updating – Is it possible to represent the problem of KG updating as an instance of CP? If so, does this lend itself to a convenient way of dynamically updating a KG?
- CP-Bench: Evaluating Large Language Models for Constraint Modelling(https://arxiv.org/abs/2506.06052)– paper that explores the use of LLMs in constraint programming
- Automated streamliner portfolios for constraint satisfaction problems (https://www.sciencedirect.com/science/article/pii/S0004370223000619?via%3Dihub)– Streamliner strategy to reduce the constraint search space
- Enhancing constraint programming via supervised learning for job shop scheduling (https://www.sciencedirect.com/science/article/pii/S0950705124003332?)– CP + supervised learning

Disclaimer 1: It's possible that some of these projects are more difficult than I have estimated; reach out to me or do a bit of preliminary research to make sure that you have some idea of where to begin before committing to a project.
Disclaimer 2: You have learned the foundations to build some very interesting projects. Sure, some of these project ideas may seem frightening at the moment, but you *will* have to do your own research to learn new techniques. Remember, choose a project that you are truly excited about so that this doesn't feel like a chore.